

TỐI ƯU HÓA HIỆU NĂNG ỨNG DỤNG REACTJS

Đề tài: Ứng dụng quản lý 10.000 sản phẩm

Học phần: Lập trình Web Nâng Cao

Sinh viên thực hiện

Đỗ Văn Minh - B23DCCC109

Lớp

RIPT1411-20261-02

Giảng viên hướng dẫn

TS.Ngô Văn Nhận

LỜI MỞ ĐẦU

Trong các ứng dụng web quản lý như ERP, CRM và hệ thống quản trị thương mại điện tử, danh sách dữ liệu lớn là một tình huống phổ biến. Khi số lượng bản ghi tăng lên hàng nghìn hoặc hàng chục nghìn, việc kết xuất đồng thời quá nhiều phần tử giao diện có thể làm tăng chi phí xử lý JavaScript, tạo DOM, tính toán bố cục và cập nhật giao diện. Những vấn đề này ảnh hưởng trực tiếp đến tốc độ hiển thị cũng như khả năng tương tác của người dùng.

Báo cáo trình bày quá trình xây dựng và tối ưu ứng dụng quản lý 10.000 sản phẩm bằng ReactJS. Các kỹ thuật trọng tâm gồm Virtualization, Memoization, Code-splitting và useDeferredValue. Hiệu năng được đánh giá bằng Google Lighthouse trên bản production build, qua đó so sánh các chỉ số trước và sau tối ưu.

Lưu ý về số liệu: các kết quả định lượng trong báo cáo được tổng hợp theo dữ liệu kiểm thử do đề bài cung cấp. Khi nộp báo cáo, cần đối chiếu lại với các tệp Lighthouse HTML/JSON được tạo từ lần chạy thực tế; kết quả có thể thay đổi theo cấu hình máy, phiên bản trình duyệt và điều kiện kiểm thử.

MỤC LỤC

LỜI MỞ ĐẦU.....	2
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	5
1.1. Đặt vấn đề.....	5
1.2. Mục tiêu.....	5
1.3. Phạm vi và công nghệ.....	5
CHƯƠNG 2. PHÂN TÍCH ỨNG DỤNG TRƯỚC TỐI ƯU.....	6
2.1. Chức năng hệ thống.....	6
2.2. Các điểm nghẽn hiệu năng.....	6
2.2.1. Số lượng DOM quá lớn.....	6
2.2.2. Tính toán lặp lại.....	6
2.2.3. Render lại các component con.....	6
2.2.4. Bundle ban đầu chứa cả chức năng ít dùng.....	7
2.3. Tác động đến người dùng.....	7
CHƯƠNG 3. GIẢI PHÁP TỐI ƯU HIỆU NĂNG.....	8
3.1. Virtualization (Windowing).....	8
3.2. Memoization.....	9
3.2.1. useMemo.....	9
3.2.2. React.memo.....	9
3.2.3. useCallback.....	10
3.3. Code-splitting với React.lazy và Suspense.....	10
3.4. useDeferredValue.....	10

3.5. Tổng hợp giải pháp.....	11
CHƯƠNG 4. PHƯƠNG PHÁP KIỂM THỬ.....	12
4.1. Môi trường kiểm thử.....	12
4.2. Quy trình tái hiện.....	12
4.3. Ý nghĩa các chỉ số.....	12
CHƯƠNG 5. KẾT QUẢ VÀ PHÂN TÍCH.....	14
5.1. Phân tích TBT.....	14
5.2. Phân tích LCP và Speed Index.....	15
5.3. Phân tích CLS.....	15
5.4. Phân tích DOM và trải nghiệm tìm kiếm.....	15
5.5. Giới hạn và nguy cơ sai lệch.....	15
5.6. Một số hình ảnh đo.....	16
CHƯƠNG 6. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY.....	17
6.1. Cài đặt dependencies.....	17
6.2. Chạy môi trường phát triển.....	17
6.3. Build và lint.....	17
6.4. Chạy Lighthouse.....	17
CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	18
7.1. Kết luận.....	18
TÀI LIỆU THAM KHẢO.....	19

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề

Trong giao diện quản trị dữ liệu, người dùng thường xuyên tìm kiếm, lọc, sắp xếp và mở thông tin chi tiết. Nếu toàn bộ 10.000 sản phẩm cùng được kết xuất thành các phần tử DOM, trình duyệt phải xử lý lượng lớn node và công việc liên quan đến style, layout, paint. Đồng thời, các thao tác lọc và sắp xếp có thể chạy lại nhiều lần khi component cập nhật, khiến thao tác gõ tìm kiếm thiếu phản hồi.

Vì vậy, bài toán không chỉ là rút ngắn thời gian tải trang mà còn phải giảm công việc không cần thiết trong quá trình tương tác, đồng thời bảo đảm kết quả hiển thị chính xác.

1.2. Mục tiêu

- Xây dựng giao diện quản lý danh mục gồm 10.000 sản phẩm với các trường dữ liệu phong phú.
- Hỗ trợ tìm kiếm, lọc theo nhiều tiêu chí, sắp xếp và xem thông tin chi tiết.
- Áp dụng Virtualization để chỉ kết xuất các sản phẩm cần thiết trong vùng nhìn.
- Áp dụng useMemo, React.memo và useCallback để hạn chế tính toán và render lặp lại.
- Tách các chức năng ít dùng thành các chunk độc lập bằng React.lazy và Suspense.
- Sử dụng useDeferredValue để cải thiện độ phản hồi khi nhập từ khóa.
- So sánh hiệu năng trước và sau tối ưu bằng Lighthouse và kiểm tra lại kết quả trong báo cáo đầu ra.

1.3. Phạm vi và công nghệ

Hạng mục	Mô tả
Ứng dụng	Trang quản trị danh mục 10.000 sản phẩm
Thư viện giao diện	React 19
Công cụ build	Vite 8
Đo kiểm	Google Lighthouse CLI 13.5.0, Chrome Testing Engine v153.0 theo thông tin bài kiểm thử
Môi trường	Production build được phục vụ bằng vite preview, cổng 4173
Chỉ số	Performance score, FCP, LCP, TBT, CLS, Speed Index; bổ sung số DOM và cảm nhận khi tìm kiếm

CHƯƠNG 2. PHÂN TÍCH ỨNG DỤNG TRƯỚC TỐI ƯU

2.1. Chức năng hệ thống

Mỗi sản phẩm có các trường thông tin gồm mã SKU, tên, danh mục, thương hiệu, đơn giá, giá gốc, đánh giá sao, số lượng tồn kho, thẻ phân loại và mô tả. Giao diện cung cấp các chức năng:

- Tìm kiếm theo tên sản phẩm, SKU hoặc tags.
- Lọc theo 8 danh mục và 12 thương hiệu.
- Lọc trạng thái tồn kho: còn hàng, sắp hết hoặc hết hàng.
- Sắp xếp theo giá, đánh giá, tồn kho, tên hoặc ID.
- Mở Analytics Dashboard để xem biểu đồ và số liệu thống kê.
- Mở Product Detail Modal để xem hoặc cập nhật thông tin sản phẩm.

2.2. Các điểm nghẽn hiệu năng

2.2.1. Số lượng DOM quá lớn

Nếu dùng `products.map(...)` để tạo giao diện cho toàn bộ dữ liệu, mỗi sản phẩm sinh ra nhiều phần tử con như nhãn SKU, tên, danh mục, tags, giá, trạng thái kho, đánh giá và nút thao tác. Theo ước tính của bài toán, tổng số DOM node có thể vượt 120.000. Số node lớn làm tăng chi phí tạo DOM, tính toán kiểu, bố cục và vẽ lại.

2.2.2. Tính toán lặp lại

Nếu logic tìm kiếm, lọc, sắp xếp và thống kê chạy trực tiếp trong mỗi lần render, các phép tính này có thể bị thực hiện lại dù dữ liệu đầu vào liên quan không đổi. Điều đó làm tăng tải CPU khi mở modal, thay đổi trạng thái giao diện hoặc cập nhật các component khác.

2.2.3. Render lại các component con

Khi component cha render lại, các callback khai báo inline thường có tham chiếu mới. Nếu component con không được memo hóa, React có thể phải thực hiện lại công việc render cho nhiều thẻ sản phẩm, kể cả khi dữ liệu hiển thị của chúng không đổi.

2.2.4. Bundle ban đầu chứa cả chức năng ít dùng

Nếu các modal phân tích và chi tiết được import tĩnh, mã JavaScript của những chức năng này có thể được đưa vào luồng tải ban đầu. Người dùng phải tải và phân tích mã ngay cả khi chưa mở các chức năng tương ứng.

2.3. Tác động đến người dùng

- Cuộn danh sách thiếu mượt mà do nhiều node và công việc layout.
- Ô tìm kiếm phản hồi chậm khi quá trình lọc và render cạnh tranh tài nguyên với thao tác nhập.
- Thời gian tải và phân tích JavaScript ban đầu tăng khi bundle chứa nhiều chức năng.
- Mức sử dụng bộ nhớ có thể tăng theo số lượng node và dữ liệu giao diện.

CHƯƠNG 3. GIẢI PHÁP TỐI ƯU HIỆU NĂNG

3.1. Virtualization (Windowing)

Virtualization chỉ tạo các phần tử giao diện đang nằm trong vùng nhìn và một số phần tử đệm (overscan), thay vì kết xuất toàn bộ 10.000 sản phẩm. Phần dữ liệu vẫn có thể chứa đầy đủ 10.000 bản ghi, nhưng DOM chỉ chứa một tập con nhỏ phù hợp với viewport.

Với chiều cao dòng cố định `itemHeight = 76 px`, chỉ số đầu và cuối của vùng cần kết xuất có thể được xác định như sau:

```
startIndex = max(0, floor(scrollTop / itemHeight) - overscan)

endIndex = min(
  N - 1,
  floor((scrollTop + containerHeight) / itemHeight) + overscan
)

totalHeight = N * itemHeight
```

Trong đó `N` là số sản phẩm sau khi áp dụng bộ lọc; `scrollTop` là vị trí cuộn; `containerHeight` là chiều cao vùng danh sách; `overscan` là số dòng đệm ở mỗi phía. Container có chiều cao tổng thể `totalHeight` tạo thành cuộn tương ứng với toàn bộ danh sách. Các phần tử đang hiển thị được đặt đúng vị trí, thường thông qua transform hoặc vị trí top.

Hook `useVirtualizer` được mô tả là theo dõi cuộn và thay đổi kích thước bằng các cơ chế như `requestAnimationFrame` và `ResizeObserver`. Cần bảo đảm hủy listener/observer khi component unmount, tránh cập nhật state liên tục ngoài nhịp khung hình, và đồng bộ chính xác chiều cao dòng với CSS.

Hiệu quả kỳ vọng: số DOM node giảm mạnh, từ mức trên 120.000 node xuống khoảng 200 node theo số liệu bài kiểm thử. Con số thực tế phụ thuộc số cột, phần tử con, overscan và cấu trúc component.

3.2. Memoization

3.2.1. useMemo

useMemo ghi nhớ kết quả tính toán giữa các lần render khi các dependency không đổi. Trong ứng dụng, kỹ thuật này được dùng cho danh sách đã lọc/sắp xếp và số liệu thống kê kho hàng.

```
const memoizedFilteredProducts = useMemo(
  () => filterAndSortLogic(effectiveSearch),
  [
    products,
    effectiveSearch,
    filter.category,
    filter.brand,
    filter.stockFilter,
    filter.sortBy
  ]
);

const memoizedStats = useMemo(
  () => calculateInventoryStats(products, false),
  [products]
);
```

Cần bảo đảm mọi giá trị được đọc trong callback và có ảnh hưởng đến kết quả đều được đưa vào dependency array. useMemo không làm phép tính nhanh hơn trong lần đầu chạy; nó tránh tính lại khi dependency giữ nguyên.

3.2.2. React.memo

React.memo cho phép bỏ qua render lại component con khi props không đổi theo phép so sánh. Ví dụ ProductCardMemo dùng so sánh tùy biến các trường id, price, stock, rating và style.top.

```
export const ProductCardMemo = React.memo(
  ProductCardRaw,
  (prevProps, nextProps) =>
    prevProps.product.id === nextProps.product.id &&
    prevProps.product.price === nextProps.product.price &&
    prevProps.product.stock === nextProps.product.stock &&
    prevProps.product.rating === nextProps.product.rating &&
    prevProps.style?.top === nextProps.style?.top
);
```

Hàm so sánh tùy biến phải xét tất cả props có ảnh hưởng đến giao diện hoặc hành vi. Nếu bỏ sót trường như tên, tags, callback hoặc trạng thái đang chọn, component có thể hiển thị dữ liệu cũ. Vì vậy, cần kiểm thử việc cập nhật sản phẩm và không mặc định rằng so sánh ít trường luôn tốt hơn.

3.2.3. `useCallback`

`useCallback` giữ ổn định tham chiếu hàm giữa các lần render khi dependency không đổi. Các handler như `handleSelectProduct`, `handleFilterChange`, `handleResetFilter` và `getScrollElement` có thể được memo hóa khi được truyền xuống component con đã dùng `React.memo`. Cần tránh lạm dụng `useCallback` ở những vị trí không có lợi ích đo được.

3.3. Code-splitting với `React.lazy` và `Suspense`

Các màn hình ít dùng như `AnalyticsModal` và `ProductDetailModal` được tách thành các chunk JavaScript riêng. `React.lazy` tải component khi cần và `Suspense` hiển thị giao diện chờ trong lúc chunk đang được tải.

```
const LazyAnalyticsModal = React.lazy(  
  () => import('./components/AnalyticsModal')  
);  
  
const LazyProductDetailModal = React.lazy(  
  () => import('./components/ProductDetailModal')  
);
```

Theo số liệu build được cung cấp, bundle chính khoảng 242 kB; chunk `AnalyticsModal` khoảng 4,78 kB và `ProductDetailModal` khoảng 2,84 kB. Cần xác minh các kích thước này là trước hay sau gzip/brotli và có cùng cách đo với nhau hay không. Code-splitting chủ yếu giảm lượng mã cần tải/phân tích ban đầu; nó không tự động giảm chi phí render danh sách.

3.4. `useDeferredValue`

`useDeferredValue` cho phép một phần giao diện phụ thuộc vào giá trị mới cập nhật chậm hơn khi React cần ưu tiên tương tác khẩn cấp. Trong trường hợp này, giá trị nhập trực tiếp được giữ ở trạng thái ưu tiên cao, còn danh sách kết quả sử dụng giá trị trì hoãn.

```
const deferredSearch = useDeferredValue(filter.search);  
const effectiveSearch =
```

```
mode === 'optimized' ? deferredSearch : filter.search;
```

Kỹ thuật này có thể giúp ô nhập tiếp tục phản hồi trong khi danh sách lớn được cập nhật. Tuy nhiên, `useDeferredValue` không làm phép lọc tự thân nhanh hơn và không bảo đảm cố định 60 FPS. Hiệu quả phụ thuộc vào cấu trúc component, khả năng interrupt render và chi phí tính toán; nên xác nhận bằng profiling hoặc đo tương tác thực tế.

3.5. Tổng hợp giải pháp

Kỹ thuật	Vấn đề xử lý	Lợi ích chính	Điểm cần lưu ý
Virtualization	Quá nhiều DOM node	Giảm mạnh số phần tử được kết xuất cùng lúc	Đồng bộ chiều cao dòng, scroll và overscan
<code>useMemo</code>	Tính toán lặp	Tái sử dụng kết quả khi dependency không đổi	Dependency phải đầy đủ và chính xác
<code>React.memo</code>	Component con render lại không cần thiết	Bỏ qua render khi props tương đương	So sánh tùy biến không được bỏ sót props
<code>useCallback</code>	Callback thay đổi tham chiếu	Hỗ trợ memo hóa component con	Chỉ dùng khi có lợi ích thực tế
<code>React.lazy</code> + <code>Suspense</code>	Bundle đầu chứa mã ít dùng	Tải chunk theo nhu cầu	Có trạng thái loading và xử lý lỗi tải chunk
<code>useDeferredValue</code>	Nhập liệu bị cạnh tranh với cập nhật danh sách	Ưu tiên cập nhật giá trị nhập	Không thay thế tối ưu thuật toán lọc

CHƯƠNG 4. PHƯƠNG PHÁP KIỂM THỬ

4.1. Môi trường kiểm thử

Thành phần	Thiết lập được nêu trong đề bài
Ứng dụng	Bản production build của React 19 + Vite 8
Web server	vite preview, cổng 4173
Công cụ đo	Google Lighthouse CLI 13.5.0
Engine/trình duyệt	Chrome Testing Engine v153.0, chế độ Desktop
Chế độ so sánh	unoptimized và optimized qua query parameter hoặc HUD
Đầu ra	unoptimized.report.html, optimized.report.html, summary.json

4.2. Quy trình tái hiện

1. Cài đặt dependencies trong thư mục dự án.
2. Chạy npm run build để tạo bản production và kiểm tra lỗi build.
3. Chạy npm run lint để kiểm tra chất lượng mã nguồn.
4. Khởi chạy bản preview và xác nhận ứng dụng hoạt động tại cổng 4173.
5. Chạy npm run lighthouse để đo hai chế độ với cùng cấu hình.
6. Mở hai tệp HTML và summary.json để xác minh chỉ số; lưu lại phiên bản trình duyệt, cấu hình máy và thời điểm đo.
7. Lặp lại phép đo nhiều lần trong điều kiện tương tự; báo cáo giá trị trung vị hoặc nêu rõ quy tắc tổng hợp.

4.3. Ý nghĩa các chỉ số

Chỉ số	Ý nghĩa	Cách diễn giải
Performance score	Điểm tổng hợp Lighthouse	Điểm cao là tín hiệu tốt nhưng không thay thế việc xem từng metric
FCP	Thời điểm nội dung đầu tiên xuất hiện	Phản ánh thời gian người dùng bắt đầu thấy phản hồi trực quan
LCP	Thời điểm phần tử nội dung lớn nhất được vẽ	Phản ánh tốc độ hiển thị nội dung chính
TBT	Tổng thời gian các long task chặn luồng chính trong giai đoạn đo	Giá trị thấp thường cho thấy ít tác vụ dài gây cản trở hơn
CLS	Mức độ dịch chuyển bố cục ngoài dự kiến	Giá trị thấp thể hiện bố cục ổn định
Speed Index	Tốc độ nội dung hiển thị trực quan trong quá trình tải	Giá trị thấp hơn thường tốt hơn

Lưu ý phương pháp: Lighthouse chủ yếu đánh giá quá trình tải trang trong kịch bản audit. Nó không trực tiếp chứng minh rằng thao tác gõ tìm kiếm luôn đạt 60 FPS. Để đánh giá tương tác, nên bổ sung Chrome DevTools Performance, React Profiler hoặc phép đo thời gian phản hồi khi tìm kiếm.

CHƯƠNG 5. KẾT QUẢ VÀ PHÂN TÍCH

Bảng dưới đây trình bày số liệu được cung cấp trong mô tả bài tập. Các giá trị cần được kiểm chứng trực tiếp với báo cáo Lighthouse thực tế trước khi nộp.

Chỉ số	Trước tối ưu	Sau tối ưu	Nhận xét
Performance	60/100	100/100	Tăng 40 điểm theo số liệu đầu vào
FCP	0,3 s	0,3 s	Gần như không đổi; không nên ghi cải thiện 0,4% nếu hai giá trị hiển thị đều là 0,3 s mà không có số liệu gốc chính xác
LCP	0,6 s	0,3 s	Giảm khoảng 50% theo số đã làm tròn; tỷ lệ 48,2% cần số liệu gốc
TBT	37.560 ms	0 ms	Giảm rất lớn theo kết quả được cung cấp; cần xác minh audit và môi trường đo
CLS	0,000	0,000	Không ghi nhận dịch chuyển bố cục trong các lần đo được nêu
Speed Index	12,3 s	0,3 s	Giảm khoảng 97,6% theo số đã làm tròn; cần kiểm tra giá trị gốc
DOM node	Khoảng 120.000	Khoảng 200	Giảm xấp xỉ 99,83% theo ước tính đầu vào
Gỡ tìm kiếm	Lag/khựng	Mượt hơn	Nhận định định tính; cần đo thêm bằng công cụ profiling để lượng hóa

5.1. Phân tích TBT

Theo số liệu đầu vào, TBT giảm từ 37.560 ms xuống 0 ms. Đây là thay đổi rất lớn, nhưng cần diễn giải thận trọng: Virtualization giảm đáng kể số node phải tạo và cập nhật, song không thể kết luận rằng nó là nguyên nhân duy nhất khiến TBT về 0 ms. TBT còn phụ thuộc lượng JavaScript chạy trên luồng chính, tác vụ dài, môi trường audit và kịch bản Lighthouse. Cần kiểm tra trace và phần Opportunities/Diagnostics trong báo cáo HTML để xác định nguồn gây chặn.

5.2. Phân tích LCP và Speed Index

Việc chỉ kết xuất các dòng cần thiết giúp giảm công việc giao diện trước mắt và có thể cải thiện quá trình hiển thị trực quan. Tuy vậy, LCP còn chịu ảnh hưởng bởi thời gian phản hồi máy chủ, tài nguyên chặn render, kích thước bundle và phần tử LCP cụ thể. Speed Index là chỉ số tổng hợp quá trình hiển thị, không nên diễn giải đơn giản là thời gian vẽ xong toàn bộ nội dung.

5.3. Phân tích CLS

CLS bằng 0 ở cả hai chế độ cho thấy các lần audit được báo cáo không ghi nhận dịch chuyển bố cục đáng kể. Container có chiều cao tổng thể và dòng có kích thước cố định góp phần duy trì vùng cuộn ổn định. Tuy nhiên, chỉ số CLS bằng 0 trong một lần đo không bảo đảm mọi tương tác hoặc mọi thiết bị đều không xảy ra layout shift.

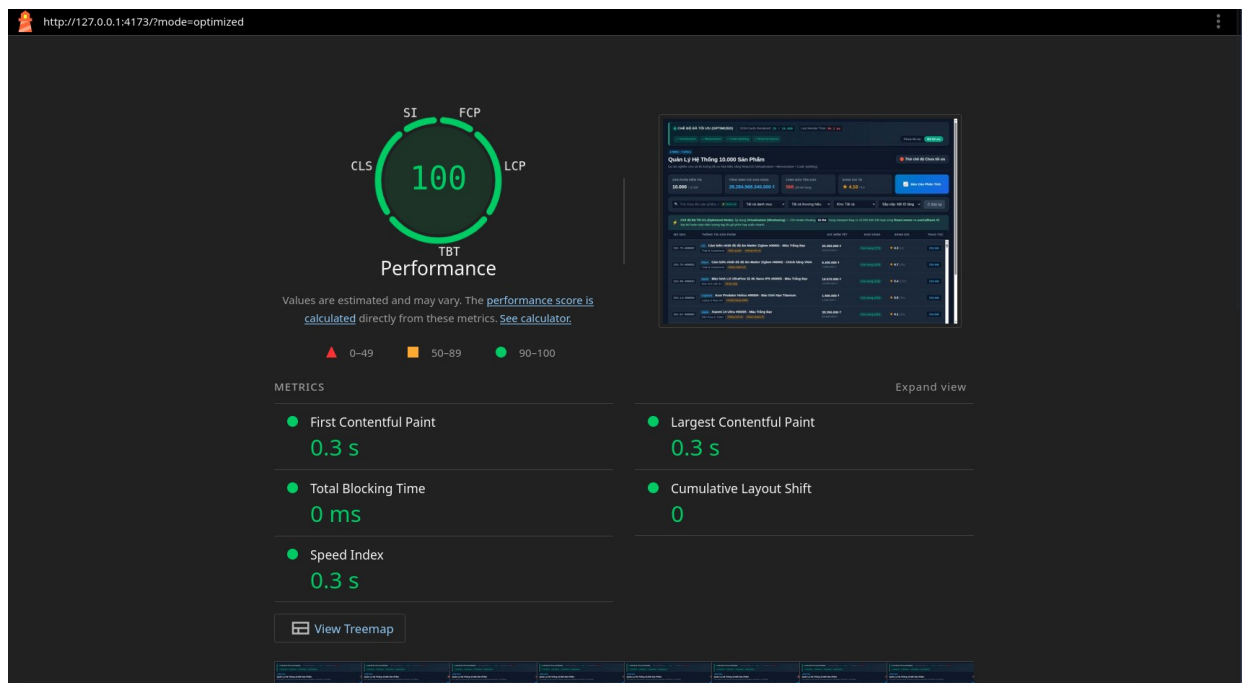
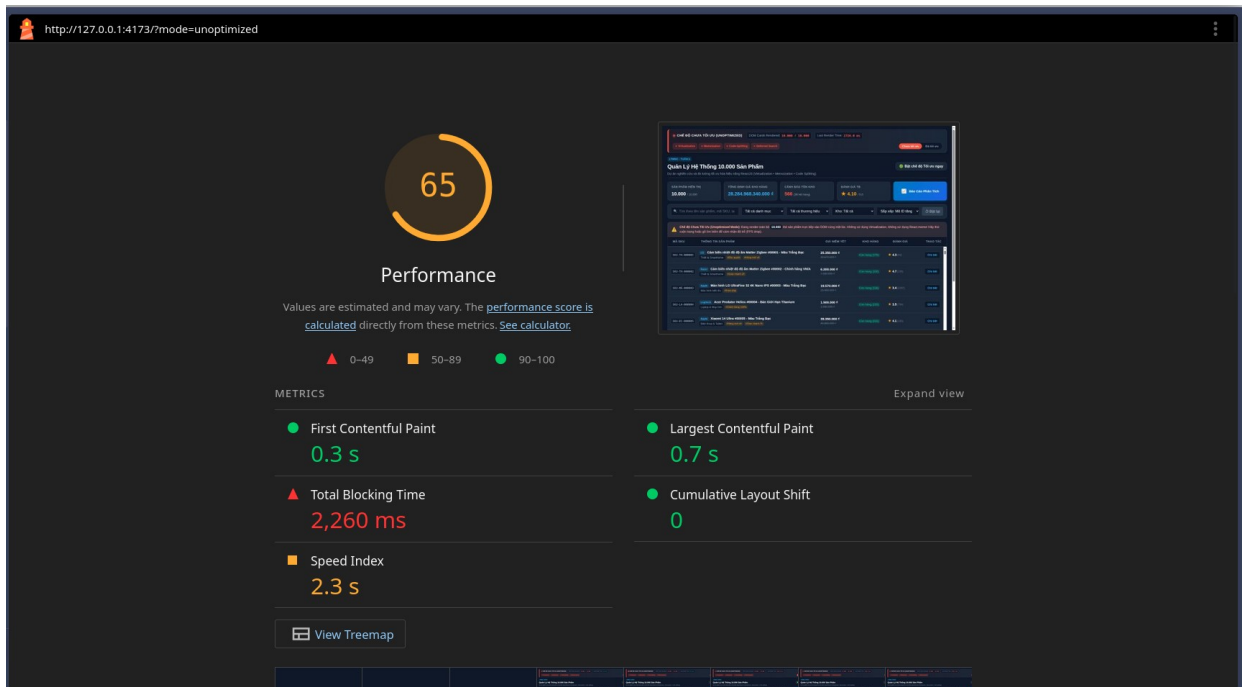
5.4. Phân tích DOM và trải nghiệm tìm kiếm

Virtualization là giải pháp trực tiếp nhất để hạn chế số lượng node hiện diện trong DOM. Memoization giảm công việc tính toán và render dư thừa khi dependency/props không đổi. useDeferredValue giúp React trì hoãn cập nhật danh sách để ưu tiên phản hồi nhập liệu. Ba kỹ thuật bổ trợ nhau nhưng giải quyết các lớp vấn đề khác nhau.

5.5. Giới hạn và nguy cơ sai lệch

- Các số liệu Lighthouse có thể dao động theo tải hệ thống, cache, CPU, phiên bản Chrome và cấu hình audit.
- Giá trị đã làm tròn không đủ để suy ra chính xác phần trăm cải thiện; cần lấy số gốc trong JSON.
- TBT bằng 0 ms là kết quả của kịch bản đo, không có nghĩa toàn bộ ứng dụng không còn tác vụ dài trong mọi tình huống.
- Số lượng DOM node nên đo bằng cùng một phương pháp và cùng thời điểm ở hai chế độ.
- Cảm nhận mượt mà và FPS cần được kiểm tra riêng bằng trace tương tác hoặc công cụ profiling.

5.6. Một số hình ảnh đo



CHƯƠNG 6. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY

6.1. Cài đặt dependencies

```
cd BTVN/BTT5
```

```
npm install
```

6.2. Chạy môi trường phát triển

```
npm run dev
```

Mở địa chỉ <http://localhost:5173>. Có thể chuyển đổi giữa hai chế độ bằng thanh gạt trên HUD hoặc query parameter theo cấu hình ứng dụng:

```
?mode=unoptimized
```

```
?mode=optimized
```

6.3. Build và lint

```
npm run build
```

```
npm run lint
```

6.4. Chạy Lighthouse

```
npm run lighthouse
```

Script được mô tả là tự động build production, khởi chạy preview server, đo cả hai chế độ và ghi kết quả vào thư mục lighthouse-reports/. Các tệp đầu ra dự kiến gồm:

- unoptimized.report.html – báo cáo chi tiết chế độ chưa tối ưu.
- optimized.report.html – báo cáo chi tiết chế độ đã tối ưu.
- summary.json – dữ liệu tổng hợp để đối chiếu các chỉ số.

Nếu kết quả khác với bảng số liệu, cần ưu tiên số liệu thực tế trong tệp báo cáo được tạo bởi lần chạy cuối cùng, ghi rõ điều kiện kiểm thử và cập nhật lại bảng kết quả.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận

Bài tập tập trung vào một vấn đề thực tế của ứng dụng quản trị: hiển thị và tương tác với danh sách có quy mô lớn. Virtualization giúp giảm số phần tử DOM cần kết xuất; useMemo, React.memo và useCallback hạn chế tính toán hoặc render lặp lại khi điều kiện phù hợp; React.lazy và Suspense tách mã chức năng ít dùng khỏi tải ban đầu; useDeferredValue giúp ưu tiên cập nhật nhập liệu so với phần giao diện có chi phí render cao.

Theo số liệu kiểm thử được cung cấp, phiên bản tối ưu đạt điểm Lighthouse cao hơn, giảm đáng kể TBT và Speed Index, đồng thời giảm số DOM node. Kết quả cần được xác minh với báo cáo HTML/JSON thực tế để bảo đảm khả năng tái lập và tính chính xác của các tỷ lệ cải thiện.

TÀI LIỆU THAM KHẢO

- [1] React Documentation, "useMemo," *React*, 2024. [Online]. Available: <https://react.dev/reference/react/useMemo>. [Accessed: 04-Oct-2026].
- [2] React Documentation, "memo," *React*, 2024. [Online]. Available: <https://react.dev/reference/react/memo>. [Accessed: 04-Oct-2026].
- [3] React Documentation, "useCallback," *React*, 2024. [Online]. Available: <https://react.dev/reference/react/useCallback>. [Accessed: 04-Oct-2026].
- [4] React Documentation, "lazy," *React*, 2024. [Online]. Available: <https://react.dev/reference/react/lazy>. [Accessed: 04-Oct-2026].
- [5] React Documentation, "Suspense," *React*, 2024. [Online]. Available: <https://react.dev/reference/react/Suspense>. [Accessed: 04-Oct-2026].
- [6] React Documentation, "useDeferredValue," *React*, 2024. [Online]. Available: <https://react.dev/reference/react/useDeferredValue>. [Accessed: 04-Oct-2026].
- [7] Google Developers, "Optimize long tasks and Core Web Vitals," *web.dev*. [Online]. Available: <https://web.dev/>. [Accessed: 04-Oct-2026].
- [8] Chrome for Developers, "Overview of Lighthouse," *Chrome Developers Documentation*. [Online]. Available: <https://developer.chrome.com/docs/lighthouse/overview/>. [Accessed: 04-Oct-2026].